L'analyse fréquentiste par l'approche bag-of-words : intérêt et limites

Lino Galiana
Version du 2 novembre 2026

Ce chapitre approfondit l'analyse fréquentiste appliquée aux données textuelles en présentant les enjeux du domaine de l'extraction d'informations (*information retrieval*). Après avoir présenté le nettoyage et la vectorisation des textes via TF-IDF, il aborde les limites de l'approche *bag of words* et ouvre la voie aux *embeddings*, vecteurs représentant les objets textuels par proximité de sens, objet du prochain chapitre.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **2**
1.1 Données 2
2 La mesure TF-IDF (*term frequency - inverse document frequency*) **2**
2.1 La matrice documents - termes 2
2.2 Utilisation pour l'extraction d'informations 3
2.3 Exemple 4
2.4 Application 5
3 Un premier enrichissement de l'approche sac de mots : les *n-grams* **12**
4 Quelques applications **14**
4.1 Reconnaissance des entités nommées 14
4.2 Classification de données textuelles : l'algorithme `Fasttext` 15
Informations additionnelles 16
Bibliographie **18**

Pour avancer dans ce chapitre, nous avons besoin de quelques installations préalables:

```
!pip install spacy
!python -m spacy download en_core_web_sm
!python -m spacy download fr_core_news_sm
```

Il est également utile de définir la fonction suivante, issue de notre chapitre précédent :

```
def clean_text(doc):
    # Tokenize, remove stop words and punctuation, and lemmatize
    cleaned_tokens = [token.lemma_ for token in doc if not token.is_stop and not token.is_punct]
    # Join tokens back into a single string
    cleaned_text = ' '.join(cleaned_tokens)
    return cleaned_text
```

1 Introduction

Nous avons vu, précédemment, l'intérêt de nettoyer les données pour dégrossir le volume d'information présent dans nos données non structurées. L'objectif de ce chapitre est d'approfondir notre compréhension de l'approche fréquentiste appliquée aux données textuelles. Nous allons évoquer la manière dont cette analyse fréquentiste permet de synthétiser l'information présente dans un corpus textuel. Nous allons également voir la manière dont on peut raffiner l'approche *bag of words* en tenant compte de l'ordre de la proximité des termes dans une phrase.

1.1 Données

Nous allons reprendre le jeu de données anglo-saxon du chapitre précédent, à savoir des textes des auteurs fantastiques Edgar Allan Poe (*EAP*), HP Lovecraft (*HPL*) et Mary Wollstonecraft Shelley (*MWS*).

```
import pandas as pd

url='https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.csv'
#1. Import des données
horror = pd.read_csv(url,encoding='latin-1')
#2. Majuscules aux noms des colonnes
horror.columns = horror.columns.str.capitalize()
#3. Retirer le prefixe id
horror['ID'] = horror['Id'].str.replace("id","")
horror = horror.set_index('Id')
```

2 La mesure TF-IDF (*term frequency - inverse document frequency*)

2.1 La matrice documents - termes

Comme nous l'avons évoqué précédemment, nous construisons une représentation synthétique de notre corpus comme un sac de mots dans lesquels on pioche plus ou moins fréquemment des mots selon leur fréquence d'apparition. C'est bien sûr une représentation simpliste de la réalité : les séquences de mots ne sont pas une suite aléatoire indépendante de mots.

Cependant, avant d'évoquer ces enjeux, il nous reste à aller au bout de l'approche sac de mots. La représentation la plus caractéristique de ce paradigme est la matrice document-terme, principalement utilisée pour comparer des corpus. Celle-ci consiste à créer une matrice où chaque document est représenté par la présence ou l'absence des termes de notre corpus. L'idée est de compter le nombre de fois où les mots (les termes, en colonne) sont présents dans chaque phrase ou libellé (le document, en ligne). Cette matrice fournit alors une représentation numérique des données textuelles.

Considérons un corpus constitué des trois phrases suivantes :

- *« La pratique du tricot et du crochet »*
- *« Transmettre la passion du timbre »*
- *« Vivre de sa passion »*

La matrice document-terme associée à ce corpus est la suivante :

	crochet	de	du	et	la	passion	pratique	sa	timbre	transmettre	tricot	vivre
La pratique du tricot et du crochet	1	0	2	1	1	0	1	0	0	0	1	0
Transmettre sa passion du timbre	0	0	1	0	0	1	0	1	1	1	0	0
Vivre de sa passion	0	1	0	0	0	1	0	1	0	0	0	1

Chaque phrase du corpus est associée à un vecteur numérique. Par exemple, la phrase *« La pratique du tricot et du crochet »*, qui n'a pas de sens en soi pour une machine, devient un vecteur numérique intelligible pour elle égal à `[1, 0, 2, 1, 1, 0, 1, 0, 0, 0, 1, 0]`. Ce vecteur numérique est une représentation creuse (*sparse*) du langage puisque chaque document (ligne) ne comportera qu'une petite partie du vocabulaire total (l'ensemble des colonnes). Pour tous les mots qui n'apparaîtront pas dans le document, on aura des 0, d'où un vecteur *sparse*. Comme nous le verrons par la suite, cette représentation numérique diffère grandement des approches modernes d'*embeddings*, basées sur l'idée de représentation denses.

2.2 Utilisation pour l'extraction d'informations

Différents documents peuvent alors être rapprochés sur la base de ces mesures. C'est l'une des manières de procéder des moteurs de recherche même si les meilleurs utilisent des approches bien plus sophistiquées. La métrique tf-idf (*term frequency–inverse document frequency*) permet de calculer un score de proximité entre un terme de recherche et un document à partir de deux composantes :

$$\text{tf-idf}(t, d, D) = \text{tf}(t, d) \times \text{idf}(t, D)$$

avec t un terme particulier (par exemple un mot), d un document particulier et D l'ensemble de documents dans le corpus.

- La partie `tf` calcule une fonction croissante de la fréquence du terme de recherche dans le document à l'étude ;
- La partie `idf` calcule une fonction inversement proportionnelle à la fréquence du terme dans l'ensemble des documents (ou corpus).
- La première partie (*term-frequency*, TF) est la fréquence d'apparition du terme terme t dans le document d. Il existe des mesures de normalisation pour éviter de biaiser la mesure en cas de documents longs.

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

où $f_{t,d}$ est le nombre brut de fois que le terme t apparaît dans le document d et le dénominateur est le nombre de termes dans le document d.

- La seconde partie (*inverse document frequency*, IDF) mesure la rareté, ou au contraire l'aspect commun, d'un terme dans l'ensemble du corpus. Si N est le nombre total de documents dans le corpus D, cette partie de la mesure sera

$$\text{idf}(t, D) = \log\left(\frac{N}{|\{d \in D : t \in d\}|}\right)$$

Le dénominateur ($|\{d \in D : t \in d\}|$) correspond au nombre de documents contenant le terme t, s'il apparaît. Plus le mot est rare, plus sa présence dans un document sera surpondérée.

De nombreux moteurs de recherche utilisent cette logique pour rechercher les documents les plus pertinents pour répondre à des termes de recherche. C'est notamment le cas d' `ElasticSearch`, le logiciel qui permet d'implémenter des moteurs de recherche efficaces. Pour classer les documents les plus pertinents par rapport à des termes de recherche, ce logiciel utilise notamment la mesure de distance BM25 qui est une version sophistiquée de la mesure TF-IDF.

2.3 Exemple

Prenons une illustration à partir d'un petit corpus. Le code suivant implémente une mesure TF-IDF. Celle-ci est légèrement différente de celle définie ci-dessus pour s'assurer de ne pas effectuer de division par zéro.

```
import numpy as np

# Documents d'exemple
documents = [
    "Le corbeau et le renard",
    "Rusé comme un renard",
    "Le chat est orange comme un renard"
]

# Tokenisation
def preprocess(doc):
    return doc.lower().split()

tokenized_docs = [preprocess(doc) for doc in documents]

# Term frequency (TF)
def term_frequency(term, tokenized_doc):
    term_count = tokenized_doc.count(term)
    return term_count / len(tokenized_doc)

# Inverse document frequency (DF)
def document_frequency(term, tokenized_docs):
    return sum(1 for doc in tokenized_docs if term in doc)

# Calculate inverse document frequency (IDF)
def inverse_document_frequency(word, corpus):
    # Normalisation avec + 1 pour éviter la division par zéro
    count_of_documents = len(corpus) + 1
    count_of_documents_with_word = sum([1 for doc in corpus if word in doc]) + 1
    idf = np.log10(count_of_documents/count_of_documents_with_word) + 1
    return idf

# Calculate TF-IDF scores in each document
def tf_idf_term(term):
  tf_idf_scores = pd.DataFrame(
    [
      [
      term_frequency(term, doc),
      inverse_document_frequency(term, tokenized_docs)
      ] for doc in tokenized_docs
    ],
    columns = ["TF", "IDF"]
  )
  tf_idf_scores["TF-IDF"] = tf_idf_scores["TF"] * tf_idf_scores["IDF"]
  return tf_idf_scores
```

Commençons par calculer TF-IDF du mot « chat » pour chaque document. De manière naturelle, c'est le troisième document, le seul où apparaît le mot qui a la valeur maximale :

```
tf_idf_term("chat")
```

	TF	IDF	TF-IDF
0	0.000000	1.30103	0.000000
1	0.000000	1.30103	0.000000
2	0.142857	1.30103	0.185861

Qu'en est-il du terme renard qui apparaît dans tous les documents (dont la partie idf est donc égale à 1) ? Dans ce cas, c'est le document où le mot est le plus fréquent, en l'occurrence le 2e, qui a la mesure maximale.

```
tf_idf_term("renard")
```

	TF	IDF	TF-IDF
0	0.200000	1.0	0.200000
1	0.250000	1.0	0.250000
2	0.142857	1.0	0.142857

2.4 Application

L'exemple précédent ne passait pas très bien à l'échelle. Heureusement, `Scikit` propose une implémentation de la recherche par vecteur TF-IDF que nous pouvons explorer avec un nouvel exercice.

Exercice 1 : TF-IDF : calcul de fréquence

1. Utiliser le vectoriseur TF-IdF de `scikit-learn` pour transformer notre corpus en une matrice `document x terms`. Au passage, utiliser l'option `stop_words` pour ne pas provoquer une inflation de la taille de la matrice. Nommer le modèle `tfidf` et le jeu entraîné `tfs`.
2. Après avoir construit la matrice de documents x terms avec le code suivant, rechercher les lignes où les termes ayant la structure `abandon` sont non-nuls.
3. Trouver les 50 extraits où le score TF-IDF du mot *« fear »* est le plus élevé et l'auteur associé. Déterminer la répartition des auteurs dans ces 50 documents.
4. Observer les 10 scores où TF-IDF de *« fear »* sont les plus élevés

Aide pour la question 2

```
feature_names = tfidf.get_feature_names_out()
corpus_index = [n for n in list(tfidf.vocabulary_.keys())]
horror_dense = pd.DataFrame(tfs.todense(), columns=feature_names)
```

Le vectoriseur obtenu à l'issue de la question 1 est le suivant :

	stop_words **stop_words: {'english'}, list, default=None** **If a string, it is passed to _check_stop_list and the appropriate stop** **list is returned. 'english' is currently the only supported string** **value.** **There are several known issues with 'english' and you should**	['just', 'other', ...]

	consider an alternative (see :ref:`stop_words`). **If a list, that list is assumed to contain stop words, all of which** **will be removed from the resulting tokens.** **Only applies if ``analyzer == 'word'``.** **If None, no stop words will be used. In this case, setting `max_df`** **to a higher value, such as in the range (0.7, 1.0), can automatically detect** **and filter stop words based on intra corpus document frequency of terms.**	
	input input: {'filename', 'file', 'content'}, default='content' - If `'filename'`, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze. - If `'file'`, the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory. - If `'content'`, the input is expected to be a sequence of items that can be of type string or byte.	'content'
	encoding encoding: str, default='utf-8' If bytes or files are given to analyze, this encoding is used to decode.	'utf-8'
	decode_error decode_error: {'strict', 'ignore', 'replace'}, default='strict' Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given `encoding`. By default, it is	'strict'

	'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.	
	strip_accents strip_accents: {'ascii', 'unicode'} or callable, default=None Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed. Both 'ascii' and 'unicode' use NFKD normalization from :func:`unicodedata.normalize`.	None
	lowercase lowercase: bool, default=True Convert all characters to lowercase before tokenizing.	True
	preprocessor preprocessor: callable, default=None Override the preprocessing (string transformation) stage while preserving the tokenizing and n-grams generation steps. Only applies if ``analyzer`` is not callable.	None
	tokenizer tokenizer: callable, default=None Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if ``analyzer == 'word'``.	None
	analyzer analyzer: {'word', 'char', 'char_wb'} or callable, default='word' Whether the feature should be made of word or character n-grams. Option 'char_wb' creates cha-	'word'

	racter n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space. If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input. .. versionchanged:: 0.21 Since v0.21, if ``input`` is ``'filename'`` or ``'file'``, the data is first read from the file and then passed to the given callable analyzer.	
	token_pattern token_pattern: str, default=r"(?u)\\b\\w\\w+\\b" Regular expression denoting what constitutes a "token", only used if ``analyzer == 'word'``. The default regexp selects tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator). If there is a capturing group in token_pattern then the captured group content, not the entire match, becomes the token. At most one capturing group is permitted.	'(?u)\\b\\w\\w+\\b'
	ngram_range ngram_range: tuple (min_n, max_n), default=(1, 1) The lower and upper boundary of the range of n-values for different n-grams to be extracted. All values of n such that min_n <= n <= max_n will be used. For example an ``ngram_range`` of ``(1, 1)`` means only unigrams, ``(1, 2)`` means unigrams and bigrams, and ``(2, 2)`` means	(1, ...)

	only bigrams. Only applies if ``analyzer`` is not callable.	
	max_df max_df: float or int, default=1.0 When building the vocabulary ignore terms that have a document frequency strictly higher than the given threshold (corpus-specific stop words). If float in range [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1.0
	min_df min_df: float or int, default=1 When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float in range of [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1
	max_features max_features: int, default=None If not None, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus. Otherwise, all features are used. This parameter is ignored if vocabulary is not None.	None
	vocabulary vocabulary: Mapping or iterable, default=None Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or	None

	an iterable over terms. If not given, a vocabulary is determined from the input documents.	
	binary binary: bool, default=False If True, all non-zero term counts are set to 1. This does not mean outputs will have only 0/1 values, only that the tf term in tf-idf is binary. (Set `binary` to True, `use_idf` to False and `norm` to None to get 0/1 outputs).	False
	dtype dtype: dtype, default=float64 Type of the matrix returned by fit_transform() or transform().	<class 'numpy.float64'>
	norm norm: {'l1', 'l2'} or None, default='l2' Each output row will have unit norm, either: - 'l2': Sum of squares of vector elements is 1. The cosine similarity between two vectors is their dot product when l2 norm has been applied. - 'l1': Sum of absolute values of vector elements is 1. See :func:`~sklearn.preprocessing.normalize`. - None: No normalization.	'l2'
	use_idf use_idf: bool, default=True Enable inverse-document-frequency reweighting. If False, idf(t) = 1.	True
	smooth_idf smooth_idf: bool, default=True Smooth idf weights by adding one to document frequencies, as if an extra document was seen containing every term in the collection exactly once. Prevents zero divisions.	True

sublinear_tf	sublinear_tf: bool, default=False Apply sublinear tf scaling, i.e. replace tf with 1 + log(tf).

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/sklearn/
feature_extraction/text.py:412: UserWarning: Your stop_words may be inconsistent with your
preprocessing. Tokenizing the stop words generated tokens ['ll', 've'] not in stop_words.
  warnings.warn(
```

	aaem	ab	aback	abaft	abandon	abandoned	abandoning	abandonment	abaout	abased	...	zodiacal	zoilus	zokkar	zone	zones	zopyrus	zorry	zubmizzion	zuro	á¼
0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.267616	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Les lignes où le terme *« abandon »* apparait sont les suivantes (question 2) :

```
Index([    4,   116,   215,   571,   839,  1042,  1052,  1069,  2247,  2317,
        2505,  3023,  3058,  3245,  3380,  3764,  3886,  4425,  5289,  5576,
        5694,  6812,  7500,  9013,  9021,  9077,  9560, 11229, 11395, 11451,
       11588, 11827, 11989, 11998, 12122, 12158, 12189, 13666, 15259, 16516,
       16524, 16759, 17547, 18019, 18072, 18126, 18204, 18251],
      dtype='int64')
```

La matrice document-terme associée à celles-ci est la suivante :

	aaem	ab	aback	abaft	abandon	abandoned	abandoning	abandonment	abaout	abased	...	zodiacal	zoilus	zokkar	zone	zones	zopyrus	zorry	zubmizzion	zuro	á¼
4	0.0	0.0	0.0	0.0	0.000000	0.267616	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
116	0.0	0.0	0.0	0.0	0.000000	0.359676	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
215	0.0	0.0	0.0	0.0	0.249090	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
571	0.0	0.0	0.0	0.0	0.000000	0.153280	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
839	0.0	0.0	0.0	0.0	0.312172	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

On remarque ici l'inconvénient de ne pas avoir fait de racinisation. Les variations de *« abandon »* sont éclatées sur de nombreuses colonnes. *« abandoned »* est aussi différent d'*« abandon »* que de *« fear »*. C'est l'un des problèmes de l'approche *bag of words*.

```
Text    50
dtype: int64
```

Les 10 scores les plus élevés sont les suivants :

```
['We could not fear we did not.',
 '"And now I do not fear death.',
 'Be of heart and fear nothing.',
 'Indeed I had no fear on her account.',
 'I smiled, for what had I to fear?',
 'I did not like everything about what I saw, and felt again the fear I had had.',
 'At length, in an abrupt manner she asked, "Where is he?" "O, fear not," she continued, "fear
not that I should entertain hope Yet tell me, have you found him?',
 'I have not the slightest fear for the result.',
 '"I fear you are right there," said the Prefect.']
```

On remarque que les scores les plus élevés sont soient des extraits courts où le mot apparait une seule fois, soit des extraits plus longs où le mot *« fear »* apparaît plusieurs fois.

3 Un premier enrichissement de l'approche sac de mots : les *n-grams*

Nous avons évoqué deux principales limites à l'approche sac de mot : l'absence de prise en compte du contexte et la représentation *sparse* du langage qui rend les rapprochements entre texte parfois moyennement pertinents. Dans le paradigme du sac de mots, il est néanmoins possible de prendre en compte la séquence d'enchainement de sèmes (*tokens*) par le biais des *ngrams*.

Pour rappel, jusqu'à présent, dans l'approche *bag of words*, l'ordre des mots n'avait pas d'importance. On considère qu'un texte est une collection de mots tirés indépendamment, de manière plus ou moins fréquente en fonction de leur probabilité d'occurrence. Cependant, tirer un mot particulier n'affecte pas les chances de tirer certains mots ensuite, de manière conditionnelle.

Une manière d'introduire des liens entre les séries de *tokens* sont les *n-grams*. On s'intéresse non seulement aux mots et à leur fréquence, mais aussi aux mots qui suivent. Cette approche est essentielle pour désambiguiser les homonymes. Le calcul de *n-grams*[1] constitue la méthode la plus simple pour tenir compte du contexte.

Pour être en mesure de mener cette analyse, il est nécessaire de télécharger un corpus supplémentaire :

```
import nltk
nltk.download('genesis')
nltk.corpus.genesis.words('english-web.txt')
```

```
[nltk_data] Downloading package genesis to /github/home/nltk_data...
[nltk_data]   Package genesis is already up-to-date!
```

```
['In', 'the', 'beginning', 'God', 'created', 'the', ...]
```

`NLTK` offre des méthodes pour tenir compte du contexte. Pour ce faire, nous calculons les n-grams, c'est-à-dire l'ensemble des co-occurrences successives de mots n-à-n. En général, on se contente de bi-grams, au mieux de tri-grams :

- les modèles de classification, analyse du sentiment, comparaison de documents, etc. qui comparent des n-grams avec n trop grands sont rapidement confrontés au problème de données sparse, cela réduit la capacité prédictive des modèles ;
- les performances décroissent très rapidement en fonction de n, et les coûts de stockage des données augmentent rapidement (environ n fois plus élevé que la base de données initiale).

On va, rapidement, regarder dans quel contexte apparaît le mot `fear` dans l'oeuvre d'Edgar Allan Poe (EAP). Pour cela, on transforme d'abord le corpus EAP en tokens `NLTK` :

```
eap_clean = horror.loc[horror["Author"] == "EAP"]
eap_clean = ' '.join(eap_clean['Text'])
tokens = eap_clean.split()
print(tokens[:10])
text = nltk.Text(tokens)
print(text)
```

```
['This', 'process,', 'however,', 'afforded', 'me', 'no', 'means', 'of', 'ascertaining', 'the']
<Text: This process, however, afforded me no means of...>
```

[1] On parle de *bigrams* pour les co-occurences de mots deux-à-deux, *trigrams* pour les co-occurences trois-à-trois, etc.

Vous aurez besoin des fonctions `BigramCollocationFinder.from_words` et `BigramAssocMeasures.likelihood_ratio` :

> Exercice 2 : n-grams et contexte du mot fear
>
> 1. Utiliser la méthode `concordance` pour afficher le contexte dans lequel apparaît le terme `fear`.
> 2. Sélectionner et afficher les meilleures collocations, par exemple selon le critère du ratio de vraisemblance.
>
> Lorsque deux mots sont fortement associés, cela est parfois dû au fait qu'ils apparaissent rarement. Il est donc parfois nécessaire d'appliquer des filtres, par exemple ignorer les bigrammes qui apparaissent moins de 5 fois dans le corpus.
>
> 3. Refaire la question précédente en utilisant toujours un modèle `BigramCollocationFinder` suivi de la méthode `apply_freq_filter` pour ne conserver que les bigrammes présents au moins 5 fois. Puis, au lieu d'utiliser la méthode de maximum de vraisemblance, testez la méthode `nltk.collocations.BigramAssocMeasures().jaccard`.
> 4. Ne s'intéresser qu'aux *collocations* qui concernent le mot *fear*

Avec la méthode `concordance` (question 1), la liste devrait ressembler à celle-ci :

```
Exemples d'occurences du terme 'fear' :
Displaying 13 of 13 matches:
d quick unequal spoken apparently in fear as well as in anger. What he said wa
hutters were close fastened, through fear of robbers, and so I knew that he co
to details. I even went so far as to fear that, as I occasioned much trouble,
years of age, was heard to express a fear "that she should never see Marie aga
ich must be entirely remodelled, for fear of serious accident I mean the steel
 my arm, and I attended her home. 'I fear that I shall never see Marie again.'
clusion here is absurd. "I very much fear it is so," replied Monsieur Maillard
bt of ultimately seeing the Pole. "I fear you are right there," said the Prefe
er occurred before.' Indeed I had no fear on her account. For a moment there w
erhaps so," said I; "but, Legrand, I fear you are no artist. It is my firm int
 raps with a hammer. Be of heart and fear nothing. My daughter, Mademoiselle M
e splendor. I have not the slightest fear for the result. The face was so far
arriers of iron that hemmed me in. I fear you have mesmerized" adding immediat
```

Même si on peut facilement voir le mot avant et après, cette liste est assez difficile à interpréter car elle recoupe beaucoup d'informations.

La `collocation` consiste à trouver les bi-grammes qui apparaissent le plus fréquemment ensemble. Parmi toutes les paires de deux mots observées, il s'agit de sélectionner, à partir d'un modèle statistique, les « meilleures ». On obtient donc avec cette méthode (question 2):

```
[('of', 'the'),
 ('in', 'the'),
 ('had', 'been'),
 ('to', 'be'),
 ('have', 'been'),
 ('I', 'had'),
 ('It', 'was'),
 ('it', 'is'),
 ('could', 'not'),
 ('from', 'the'),
 ('upon', 'the'),
 ('more', 'than'),
 ('it', 'was'),
```

```
 ('would', 'have'),
 ('with', 'a'),
 ('did', 'not'),
 ('I', 'am'),
 ('the', 'a'),
 ('at', 'once'),
 ('might', 'have')]
```

Si on modélise les meilleures collocations :

```
"Gad Fly"
'Hum Drum,'
'Rowdy Dow,'
Brevet Brigadier
BarriÃ¨re du
ugh ugh
Ourang Outang
Chess Player
John A.
A. B.
hu hu
General John
'Oppodeldoc,' whoever
mille, mille,
Brigadier General
```

Cette liste a un peu plus de sens, on a des noms de personnages, de lieux mais aussi des termes fréquemment employés ensemble (*Chess Player* par exemple).

En ce qui concerne les *collocations* du mot fear :

```
[('fear', 'of'), ('fear', 'God'), ('I', 'fear'), ('the', 'fear'), ('The', 'fear'), ('fear',
'him'), ('you', 'fear')]
```

Si on mène la même analyse pour le terme *love*, on remarque que de manière logique, on retrouve bien des sujets généralement accolés au verbe :

```
[('love', 'me'), ('love', 'he'), ('will', 'love'), ('I', 'love'), ('love', ','), ('you', 'love'),
('the', 'love')]
```

4 Quelques applications

Nous venons d'évoquer un premier cas d'application de l'approche *bag of words* qui est le rapprochement de textes par leurs termes communs. Ce n'est pas le seul cas d'application de l'approche précédente. Nous allons en évoquer deux qui nous amènent vers la modélisation du langage : la reconnaissance d'entités nommées et la classification.

4.1 Reconnaissance des entités nommées

La reconnaissance d'entités nommées, également connue sous l'acronyme NER pour *named entity recognition*, est une méthode d'extraction d'information permettant d'identifier, dans un texte, la nature de certains termes dans une certain classification : lieu, personne, quantité, etc.

Pour illustrer cela, reprenons le *Comte de Monte Cristo* et regardons sur un petit morceau de cette oeuvre ce qu'implique la reconnaissance d'entités nommées :

```
import requests
import re

url = "https://www.gutenberg.org/files/17989/17989-0.txt"
response = requests.get(url)
response.encoding = 'utf-8'  # Assure le bon décodage
raw = response.text

dumas = (
  raw
  .split("*** START OF THE PROJECT GUTENBERG EBOOK 17989 ***")[1]
  .split("*** END OF THE PROJECT GUTENBERG EBOOK 17989 ***")[0]
)


def clean_text(text):
    text = text.lower() # mettre les mots en minuscule
    text = " ".join(text.split())
    return text

dumas = clean_text(dumas)

dumas[10000:10500]
```

Ligne 13

On extrait de manière un petit peu simpliste le contenu de l'ouvrage

```
" mes yeux. --vous avez donc vu l'empereur aussi? --il est entré chez le maréchal pendant que
j'y étais. --et vous lui avez parlé? --c'est-à-dire que c'est lui qui m'a parlé, monsieur, dit
dantès en souriant. --et que vous a-t-il dit? --il m'a fait des questions sur le bâtiment, sur
l'époque de son départ pour marseille, sur la route qu'il avait suivie et sur la cargaison qu'il
portait. je crois que s'il eût été vide, et que j'en eusse été le maître, son intention eût été
de l'acheter; mais je lu"
```

```
import spacy
from spacy import displacy

nlp = spacy.load("fr_core_news_sm")
doc = nlp(dumas[15000:17000])
# displacy.render(doc, style="ent", jupyter=True)
```

La reconnaissance d'entités nommées disponible par défaut dans les librairies généralistes est souvent décevante ; il est souvent nécessaire d'enrichir les règles par défaut par des règles *ad hoc*, propres à chaque corpus.

En pratique, récemment, l'approche de reconnaissance d'entités nommées a été utilisée par Etalab pour pseudonymiser des documents administratifs. Il s'agit d'identifier certaines informations sensibles (état civil, adresse…) par reconnaissance d'entités pour les remplacer par des pseudonymes.

4.2 Classification de données textuelles : l'algorithme `Fasttext`

`Fasttext` est un réseau de neurone à une couche développé par Meta en 2016 pour faire de la classification de texte ou de la modélisation de langage. Comme nous allons pouvoir le voir, ce modèle va nous amener à faire la transition avec une modélisation plus raffinée du langage, bien que celle

de `Fasttext` soit beaucoup plus frustre que celle des grands modèles de langage (LLM). L'un des principaux cas d'utilisation de `Fasttext` est la classification supervisée de données textuelles. Il s'agit, à partir d'un texte, de déterminer sa catégorie d'appartenance. Par exemple, à partir d'un texte de chanson, s'il s'agit de rap ou de rock. Ce modèle est supervisé puisqu'il apprend à reconnnaître les *features*, en l'occurrence des morceaux de texte, qui permettent d'avoir une bonne performance de prédiction sur le jeu d'entraînement puis de test.

Le concept de *feature* peut sembler étonnant pour des données textuelles, qui sont, par essence, non structurées. Pour des données structurées, la démarche évoquée dans la partie modélisation apparaîssait naturelle : nous avions des variables observées dans nos données pour les *features* et l'algorithme de classification consistait à trouver la combinaison entre elles permettant de prédire, au mieux, le *label*. Avec des données textuelles, ce concept de *feature* observé n'est plus naturel. Il est nécessaire de le construire à partir d'un texte. L'information destructurée devient de l'information structurée. C'est là qu'interviennent les concepts que nous avons vu jusqu'à présent.

`FastText` est un *« sac de n-gram »*. Il considère donc que les *features* sont à construire à partir des mots de notre corpus mais aussi des *ngrams* à plusieurs niveaux. L'architecture générale de `FastText` ressemble à celle-ci :

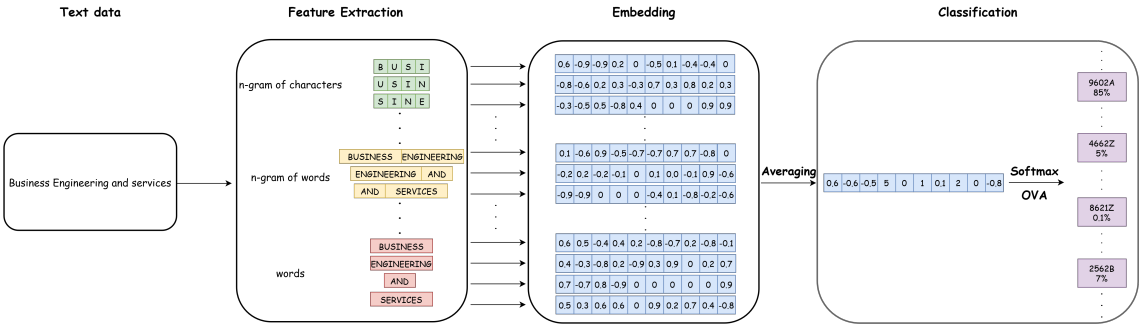


Figure 1. – Illustration de l'architecture de `FastText`

Ce qui nous intéresse ici est la partie gauche de ce diagramme, la *« feature extraction »* car la partie *embedding* correspond à des concepts que nous verrons lors des prochains chapitres. Avec l'exemple de cette figure, on voit que le texte *« Business engineering and services »* est tokenisé comme nous avons pu le voir plus tôt en mots. Mais `Fasttext` créé également des ngrams à plusieurs niveaux. Par exemple, il va créer des bigrams de mots : *« Business engineering »*, *« engineering and »*, *« and services »*. mais aussi des quadrigrammes de caractères *« busi »*, *« usin »* et *« sine »*. Ensuite, `Fasttext` transformera tous ces termes en vecteurs numériques. Contrairement à ce que nous avons vu jusqu'à présent, ces vecteurs ne sont pas des fréquences d'apparition dans le corpus (principe de la matrice origine-document), ce sont des plongements de mots (*word embedding*). Nous découvrirons leurs principes dans les prochains chapitres.

Ce modèle `Fasttext` est très utilisé dans la statistique publique car de nombreuses sources de données textuelles sont à hiérarchiser dans des nomenclatures agrégées.

```
Erreur lors de l'appel API : 503 Server Error: Service Unavailable for url: https://codification-ape-test.lab.sspcloud.fr/predict?nb_echos_max=3&prob_min=0&text_feature=data%20scientist
```

Informations additionnelles

> **i** Environnement Python
>
> Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
    "gt-extras⩾0.0.8",
    "ipykernel⩾6.29.5",
    "jupyter⩾1.1.1",
    "jupyter-cache⩾1.0.0",
    "kaleido⩾0.2.1",
    "langchain-community⩾0.3.27",
    "loguru==0.7.3",
    "markdown⩾3.8",
    "nbclient⩾0.10.0",
    "nbformat⩾5.10.4",
    "nltk⩾3.9.1",
    "openpyxl⩾3.1.5",
    "pandas⩾3.0",
    "pip⩾25.1.1",
    "plotly⩾6.1.2",
    "plotnine⩾0.15",
    "polars⩾1.8.2",
    "py7zr⩾1.1.3",
    "pyarrow⩾17.0.0",
    "pynsee⩾0.1.8",
    "python-dotenv⩾1.0.1",
    "python-frontmatter⩾1.1.0",
    "pywaffle⩾1.1.1",
    "requests⩾2.32.3",
    "scikit-image⩾0.24.0",
    "scikit-learn⩾1.8.0",
    "scipy⩾1.13.0",
    "seaborn⩾0.13.2",
    "selenium<4.39.0",
    "spacy⩾3.8.4",
    "webdriver-manager⩾4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
```

```
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

Bibliographie